AC820 水稻叶片病害智能识别系统

基于 Zynq-7020 的软硬件协同定点卷积神经网络加速与交互终端

摘 要

水稻叶片病害的早期判别目前主要依赖植保人员的经验，或把图片上传到云端模型处理。前者难以复制，后者在田间常无稳定网络，且存在采样与回传延迟。本文实现了一台可离线运行、单手可持的叶片病害识别终端：以 AC820 开发板（Zynq-7020，xc7z020clg484-2）为核心，OV5640 采集 224×224 图像，可编程逻辑（PL）执行定点卷积神经网络，800×480 触摸屏显示结果。

网络采用折叠批归一化后的 MobileNetV2 倒残差结构：1 个 3×3 卷积头、16 个倒残差块与 1 个池化分类尾，共 2,214,502 个参数，全部量化为 Q6.10 定点（ap_fixed<16,6,AP_RND,AP_SAT>）。PL 侧只实现 4 个 HLS IP——标准卷积 conv、双模式逐点/深度卷积 pwdwconv、残差相加 shortcut 与池化全连接 fc，一次推理固定 63 次核调用，由 ARM 按层序写寄存器并轮询完成位。整机在 100 MHz 下综合实现，已约束路径 WNS +0.530 ns、WHS +0.022 ns，无违例；资源占用 LUT 72.2%、FF 45.6%、BRAM 79.6%、DSP 60.9%。实测单帧推理 278.7 ms，其中 PL 核运算 198.8 ms、主机调度与同步 80.0 ms；界面绘制由 1103.6 ms 降至 38.2 ms，预览循环由 0.584 FPS 提升到 7.50 FPS。

关键词：Zynq-7020；FPGA 硬件加速；定点量化；MobileNetV2；PYNQ；水稻病害识别

目　录

第一部分　作品概述 3
1.1 选题背景与创新点 3
1.2 功能与特性 3
1.3 应用领域 4
1.4 主要技术特点 4
1.5 主要性能指标 4
1.6 主要创新点 5
1.7 设计流程 5
第二部分　系统组成及功能说明 6
2.1 设计原理与整体功能框图 6
2.2 硬件系统介绍 9
2.3 软件系统介绍 10
2.4 软硬件划分依据与接口设计 12
第三部分　完成情况及性能参数 14
3.1 整体介绍 14
3.2 工程成果 17
3.3 特性成果 18
3.4 优化过程与优化前后对比 19
第四部分　总结 19
4.1 可扩展之处 19
4.2 心得体会 19
第五部分　大模型协作记录 21
5.1 协作模式与智能体工作流 21
5.2 原始提示词与任务下发 21
5.3 模型回答与关键决策 22
5.4 自我纠错轨迹 23
第六部分　技能包的提炼过程 24
6.1 三个技能包 24
6.2 从失败与观察到规则 24
第七部分　参考文献 25

第一部分　作品概述

1.1 选题背景与创新点

水稻叶片病害的早期判别目前靠农技人员目视或上传云端模型：前者依赖经验难以复制，后者在田间常无稳定网络，图像也要离开本地。本作品把一套 221 万参数的折叠 MobileNetV2 完整放进 AC820（Zynq-7020）的 PL——OV5640 采 224×224 图像，PL 完成定点推理，800×480 触摸屏显示结果，全程离线。选 Zynq 而非通用处理器有两个直接原因：一是确定性，单帧 63 次调用的耗时极差仅 1.3 ms；二是分工现实，ARM 侧优化前绘制一帧界面就要 1103.6 ms，本就无力再承担卷积。

与已有 FPGA 方案不同，本文不按层生成专用 IP、也不做整网流水，而是按“卷积形态”把全网归并为 4 个 IP（标准卷积、逐点/深度双模式核、残差相加、池化全连接），层间经 DDR 交换；代价是 51 次串行调用与 80.0 ms 主机开销，换来更少的 IP、更小的布线与验证负担，以及 PYNQ 与裸机共用的一套寄存器契约。

1.2 功能与特性

作品是一台完整的离线识别终端，具备以下功能：

- 六分类识别：对当前画面给出水稻健康叶片与五类病害之一的判定（标签见 3.3 节），并显示推理耗时；
- 实时预览：800×480 触摸屏上同时显示 224×224 相机窗口、最近一次结果卡、耗时与样本计数；
- 三键交互：屏幕下方 Detect（识别）、Light（补光灯）、Save（保存样本）三个按钮，支持触摸与 Jupyter 侧按钮两种操作方式；
- 样本留存：每次保存同时写出原始 BGR 帧、可直接复算的 Q10 输入与 JSON 元数据，用于离线复现与争议追溯；
- 双栈运行：同一套硬件既能由 PYNQ 2.7 的 Python 程序驱动，也能由 Vitis 2025.2 编译的裸机程序驱动；
- 自检与容错：启动前可执行 --check 只做权重与位流校验而不下载 PL；HLS 轮询 5 s、VDMA 2 s 超时后锁存故障并保留缓冲，不允许带故障

重启。

1.3 应用领域

直接场景是基层植保与合作社的田间巡检：设备离线工作，固定输入下每次推理 278.7 ms，可配合摄像头模组的补光使用，作为农技人员的辅助判据。第二个场景是育苗与温室：若配固定机位与固定光路，可以做成连续监测节点，用保存的样本回流形成本地数据闭环。

1.4 主要技术特点

（1）折叠倒残差的定点实现。批归一化在训练后折叠进卷积权重，权重以 float32 存盘、运行时乘 1024 向零截断为 int16；输入按 $q = \mathrm{trunc}((v\times 2048 - 255\times 1024)/255)$ 映射到 $[-1024, 1024]$；乘加结果为 (p+512)>>10 后饱和到 int16。整条通路没有浮点运算，主机与 PL 使用同一套整数约定。

（2）按卷积形态复用硬件。4 个 IP 覆盖 63 次调用：conv 1 次、pwdwconv 51 次（34 次逐点 + 17 次深度）、shortcut 10 次、fc 1 次。双模式核通过 MODE 位与两套指针语义切换，避免了为逐点与深度卷积各做一份硬件。

（3）带校验的接口契约。基址、偏移、寄存器宽度被写成一张表，加载 overlay 时逐项校验，任何一项不符即拒绝启动；两套软件共用这张表，因此不存在“Python 能跑、裸机跑不了”的分叉。

1.5 主要性能指标

表 1 汇总了本作品的关键量化指标。

表 1　主要性能指标

指标	数值
单帧推理	278.7 ms（中位） / 279.9 ms（P95）
其中 PL 核运算	198.8 ms（71.3%）
其中主机调度	80.0 ms（28.7%）
推理吞吐	3.59 次/s
采集单帧	133.0 ms（7.50 FPS）
界面绘制	38.2 ms（优化前 1103.6 ms）
预览循环	7.50 FPS / 5.97 FPS

指标	数值
整机资源	LUT 72.2%、FF 45.6%、BRAM 79.6%、DSP 60.9%
已约束时序	WNS +0.530 ns、WHS +0.022 ns、WPWS +3.870 ns
模型规模	2,214,502 参数、106 个权重文件、63 次调用
位流	4,045,692 B（SHA256 b0a00f00…0557）

1.6 主要创新点

（1）按“卷积形态”组织硬件：4 个 IP 支撑 63 次调用，双模式核复用于逐点与深度卷积。

（2）全整数 Q6.10 通路可逐位复算：权重、输入、乘加与池化倒数均定义到比特。

（3）双栈共用寄存器契约：基址、偏移与宽度成表并可校验，PYNQ 与裸机调用顺序一致。

（4）以“证据分级”约束交付：本轮实测、历史记录、未验证分开呈现，配门禁与负例反证。

（5）把协作过程产品化：3 个技能包与 13 个入口自检构成可复用资产。

1.7 设计流程

设计流程分五段：需求与赛题条款核对 → 网络与定点方案确定 → HLS 四核实现与 RTL 验证 → Vivado 整机与 Vitis 软件 → 板上联调与性能优化。每一段都设了“通过判据”，不满足不进入下一段：四核必须以 component.xml 齐全为通过，整机必须以 bitstream 100% 且 XSA 含 bit 为通过，板上必须以相机、LCD、触摸、推理四条链路同时可用为通过。

第二部分　系统组成及功能说明

2.1 设计原理与整体功能框图

网络为折叠 BN 的 MobileNetV2 倒残差结构：Head 是 3×3/步长 2 的标

准卷积（3→32）加深度卷积（32 通道）与逐点卷积（32→16）；主体 16 个倒残差块，每块为 1×1 扩展→3×3 深度卷积→1×1 投影，步长为 1 且输入输出通道相同时叠加残差；Tail 把 320 通道扩到 1280，再做 7×7 全局平均池化与 1280→6 全连接。共 2,214,502 个参数、106 个 float32 权重文件（Head 6、16 块各 6、Tail 4），一次推理 63 次核调用。

表 2　16 个倒残差块的通道与步长配置

序号	输入	扩展	输出	分辨率	步长	残差
0	16	96	24	112	2	否
1	24	144	24	56	1	是
2	24	144	32	56	2	否
3	32	192	32	28	1	是
4	32	192	32	28	1	是
5	32	192	64	28	2	否
6	64	384	64	14	1	是
7	64	384	64	14	1	是
8	64	384	64	14	1	是
9	64	384	96	14	1	否
10	96	576	96	14	1	是
11	96	576	96	14	1	是
12	96	576	160	14	2	否
13	160	960	160	7	1	是
14	160	960	160	7	1	是
15	160	960	320	7	1	否

定点通路上，权重加载时乘 1024 向零截断为 int16；像素按 $q = \mathrm{trunc}((v\times 2048 - 255\times 1024)/255)$ 映射到 $[-1024, 1024]$；乘加按 (p+512)>>10 就近舍入并饱和到 int16。相机通路另保留绿通道 200/256 衰减，与离线训练预处理不同，迁移时必须保留。

四个核按访存形态划分，而不是按层序：conv 把 112×112 划成 4×4 个 28×28 分块，每块按 8 个输出通道（Tm=8）与 3 个输入通道（Tn=3）完全展开，两块 3×57×57 缓冲乒乓掩盖搬运；pwdwconv 由 MODE 位切换——逐点以 196 像素（14×14）×16 通道为基本块、输出按 4 通道打包成 64 位，深度以 8 通道×7×7 窗口做三级乒乓。

两种模式下接口指针语义不同：逐点用 PBUS1 作源、PBUS2/3 作权重、PBUS4 作输出、SBUS1/2 作偏置；深度用 PBUS1/2 作源、PBUS3/4 作输出、SBUS1 作权重、SBUS2 作偏置。这是驱动层最容易写错的地方，2.4 节的表 6 把它固定下来。

shortcut 逐元素饱和相加，输入输出都是 4×16 bit 打包的 64 位字，流水线 II=1，10 次合计 4.5 ms；fc 用饱和加法树求 49 点均值（乘 21/1024 近似 1/49），再按通道做 6×1280 乘加，输出 6 个 int16 原始分数。

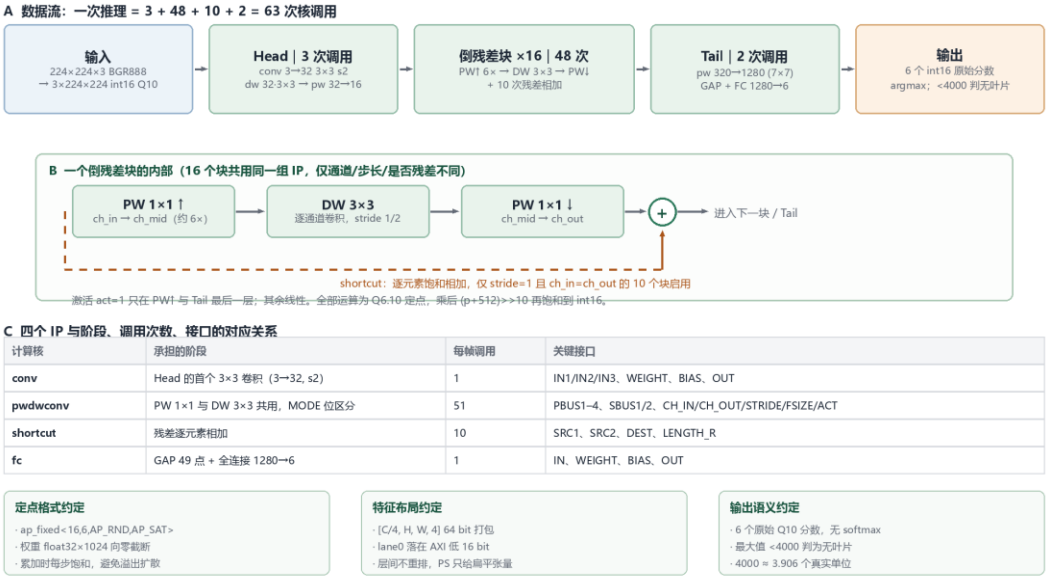


图 1　网络结构与计算核的映射

图 2 是整机框图：OV5640 以 224×224 BGR888 经 AXI4-Stream 进 VDMA 的 S2MM；LCD 800×480 由 MM2S 送出、经 RGB888→RGB565 驱动；GT911 挂 PS 的 I2C0，补光由 I2C1 写 OV5640 闪光灯寄存器。PS 负责预览、预处理、63 次调用调度与绘制，PL 为 VDMA、四个 HLS IP 与 AXI 互联，两者共享 DDR3。PL 内部的 Block Design 见图 3：PS7 + 1×VDMA + 4 个 HLS IP + 3 个 AXI 互联 + VTC。

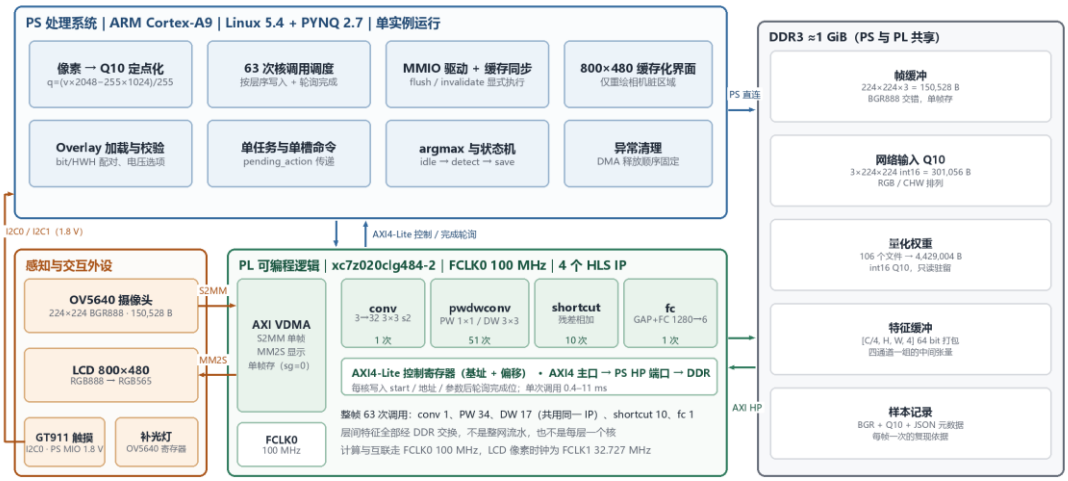


图 2　系统总体框图：PS/PL 分工、外设连接与 DDR 中的数据布局

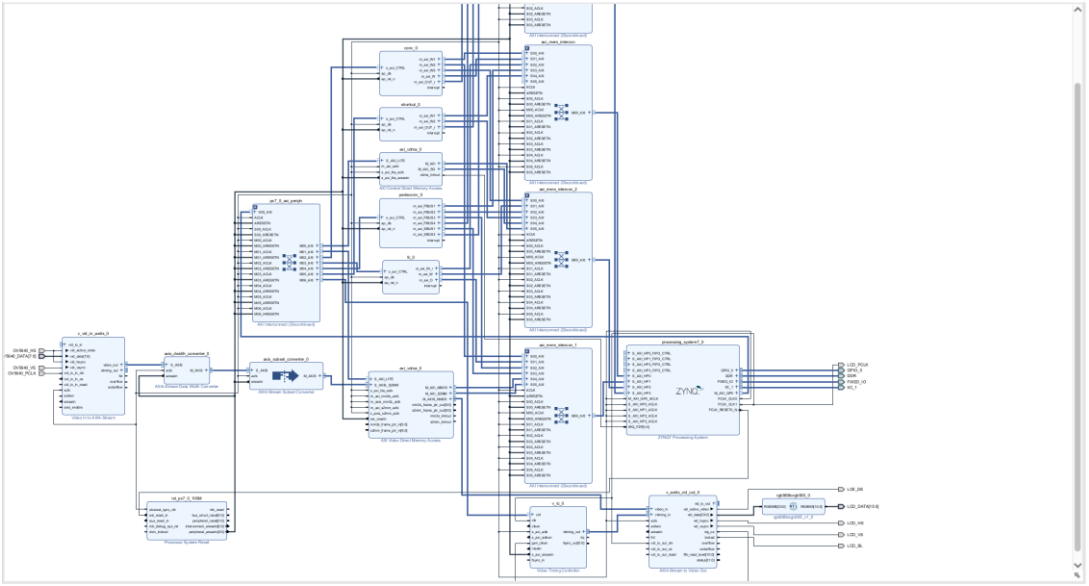


图 3　整机 Block Design（PS7 + VDMA + 4 个 HLS IP + AXI 互联 + VTC）

一次识别：取一帧（S2MM 写入并 invalidate）→ BGR 转 RGB/CHW 并量化 → 63 次核调用（写指针与标量、置 start、轮询 done）→ argmax 与阈值判断 → 重绘并 flush → MM2S 显示。全程轮询加超时，不依赖中断。

模块依赖是单向的：kernels 与 model 不 import pynq，可在主机上用假设备驱动测试；overlay 对齐 bit/HWH 与基址；lanuch 把 Notebook 的 6 段代码编译进同一命名空间并暴露 start/stop/status/request（分层见图 5）。

2.2 硬件系统介绍

表 3　平台配置

项目	配置
开发板	AC820（Zynq-7000 系列）
FPGA 器件	xc7z020clg484-2
PS	双核 ARM Cortex-A9，约 1 GiB DDR3 共享
摄像头	OV5640，I2C1（0x3C，40 kHz），AF 固件 4077 B
显示	800×480 LCD，16 位 RGB565 接口，VDMA MM2S + VTC 驱动
触摸	GT911，I2C0（0x5D，300 kHz），最多 5 点
PL 时钟	FCLK0 100 MHz（计算与互联）、FCLK1 32.727 MHz（LCD 像素时钟）
I/O 电平	PL Bank 3.3 V；触摸所用 PS MIO（MIO50/51）为 1.8 V
运行环境	PYNQ 2.7.0（Linux 5.4）；或 Vitis 2025.2 裸机

外壳为 3D 打印的上下壳，四角螺丝固定，正面开 LCD 窗、侧面留网口与 HDMI 开口（图 4）。相机与 LCD 是板载外设，经排线连接；补光取自摄像头模组自带光源（OV5640 的 0x3016/0x301C/0x3019），不另设灯组。设备无云台与自动对焦闭环，采集时由操作者持机对准叶片。

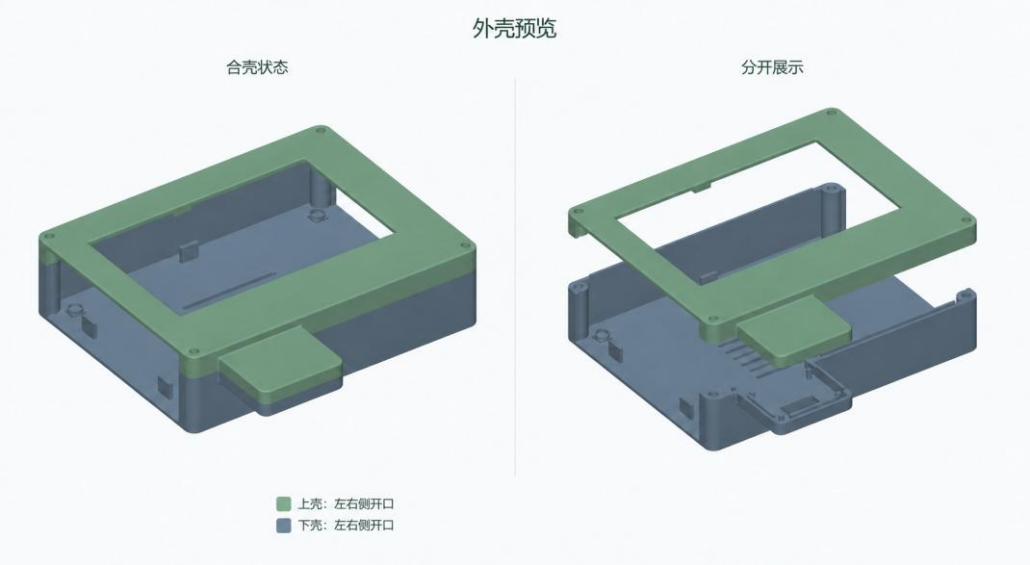


图 4　外壳设计：合壳状态与上下壳分离状态（CAD 截图）

电路的关键是三组接口的电平与方向：摄像头 8 位数据与 PCLK/HS/VS 进 PL，经 vid_in_axi4s、位宽与子集转换后进 VDMA；LCD 16 位数据与 HS/VS/DE/PCLK 由 PL 输出，统一 LVCMOS33、SLEW FAST 并上拉；触摸走 PS 的 I2C0（MIO，1.8 V），补光经 I2C1 写 OV5640 寄存器，两者都不

占 PL 的普通 IO。

表 4　主要外部接口与约束

接口	信号	电平/属性	说明
LCD 数据	LCD_DATA[15:0]	LVCMOS33，SLEW FAST，上拉	AB21/AA17/AB17/AA16 等 16 个引脚
LCD 同步	LCD_HS/LCD_VS/LCE_DE/ LCD_BL/LCD_PCLK	LVCMOS33，SLEW FAST	AB14/U14/AB19/U19/V22
摄像头数据	OV5640_DATA[7:0]	LVCMOS33	V10/R6/T4/V5/W12/ V9/T6/U4
摄像头同步	OV5640_PCLK/HS/VS	LVCMOS33	V4/AA12/AA11；PCLK 未建时钟域
触摸 I2C	IIC_1_scl_io / sda_io	LVCMOS33，上拉	AB11/AB12，PS MIO 1.8 V 域

2.3 软件系统介绍

软件分五层：入口（Notebook、lanuch CLI、infer_file）、运行时（Handle 与单槽命令）、功能（model 预处理与调度、kernels 寄存器与缓存、ui 渲染、preview 限频）、抽象（overlay 校验 bit/HWH、peripherals 封装相机/触摸/DMA）、硬件（bit+HWH、VDMA、四个 IP）。kernels 与 model 不依赖 pynq，可在主机上被假设备驱动。

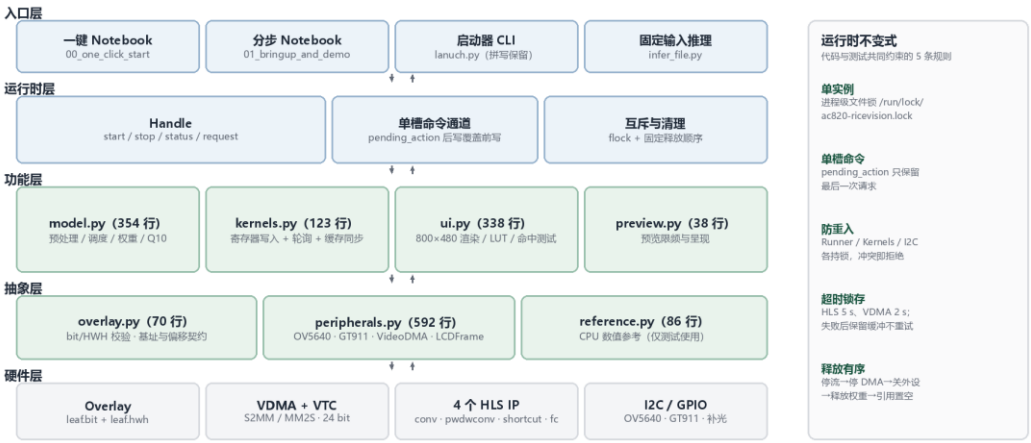


图 5　软件分层与运行时不变式

表 5　主要软件模块

模块	职责与关键实现
model.py	16 个块的配置表、weights 规格与校验、Q10 预处理、MobileNetRunner 的 63 次调用序列

模块	职责与关键实现
kernels.py	四个 IP 的寄存器写入、ap_start/ap_done 轮询、flush/invalidate 与超时锁存
peripherals.py	OV5640 初始化与单帧采集、GT911 触摸、VideoDMA 与 LCD 帧缓冲、补光控制
ui.py	800×480 界面渲染、命中测试、相机装饰的逐像素查表
preview.py	预览呈现与限频
overlay.py	bit/HWH 加载、基址与偏移契约校验
reference.py	HLS 算子的 CPU 整数参考实现（仅测试使用）
lanuch.py	进程级互斥、Notebook 单元装载、start/stop/status/request 生命周期（文件名保留原拼写）
infer_file.py	固定 BGR/Q10 输入的单次与重复推理，输出实测 JSON

程序只有一个 asyncio 循环与一个预览任务，命令经单槽变量 pending_action 传递、后写覆盖前写；三把锁（Runner、Kernels、I2C）只防重入。进程级用 /run/lock 文件锁保证只有一个实例占用 PL。

2.4 软硬件划分依据与接口设计

划分不是“卷积都放 PL”这一句话，而是四条可验证的依据：

- 算力依据：ARM 侧在优化前绘制一帧界面需要 1103.6 ms，即使优化后仍需 38.2 ms；把整网乘加也压给 A9，端到端必然进入秒级。PL 承担计算后，63 次调用合计 198.8 ms，主机只保留 80.0 ms 的调度开销。
- 确定性依据：10 次重复的单帧耗时落在 278.0—279.3 ms，极差 1.3 ms（0.5%）；这在 Linux 用户态的计算任务上很难保证。
- 复用依据：全网的卷积可分为“标准 3×3 首层”“逐点 1×1”“深度 3×3”“逐元素相加”“池化 + 全连接”五种形态，正好对应四个 IP；若按层号逐层做硬件，IP 数量与布线压力都会失控。
- 接口代价依据：层间特征经 DDR 交换，PS 每次只需写 4—12 个寄存器（逐点卷积最多）即可启动一层，单次调用平均约 1.3 ms；这比在 PL 内部做层间直连（需要为每种尺寸组合设计 FIFO 与握手）更易护，也更容易被第三方复现。

反过来，像素级预处理、界面绘制与权重校验留在 PS：它们一次性、分支多、依赖文件系统；重复且数据并行的卷积放 PL。

四个 IP 的 AXI4-Lite 基址由地址映射固定，偏移在加载 overlay 时校验。公共部分是 CTRL(0x00)、GIER(0x04)、IP_IER(0x08)、IP_ISR(0x0C)；CTRL 的 bit0 为 AP_START、bit1 为 AP_DONE、bit2 为 AP_IDLE、bit3 为 AP_READY、bit7 为 AUTO_RESTART、bit9 为 INTERRUPT。驱动在启动前先读一次 CTRL 以清掉 read-to-clear 的 done 位，写完全部指针与标量后写 1 触发，再轮询 bit1。

表 6　四个 IP 的寄存器契约

IP（基址）	偏移	字段	含义
conv 0x43C00000	0x10/0x18/0x20	in1/in2/in3	三次读入的源地址（首层三次指向同一源）
	0x28/0x30/0x38	weight/bias/out	权重、偏置、输出地址
pwdwconv 0x43C20000	0x10—0x28	PBUS1—PBUS4	逐点：源/权重/权重/输出；深度：源/源/输出/输出
	0x30/0x38	SBUS1/SBUS2	逐点：偏置/偏置；深度：权重/偏置
	0x40/0x48/0x50/0x58/0x60/0x68	ch_in/ch_out/stride/fsize/act/mode	通道、步长、特征尺寸、激活、模式
shortcut 0x43C30000	0x10/0x18/0x20	src1/src2/dest	两个加数与目的地址
	0x28	length_r	元素个数（按包计）
fc 0x43C10000	0x10/0x18/0x20/0x28	in/weight/bias/out	池化全连接的四个地址

PS 与 PL 共享 DDR，特征在层间通过 DDR 交换。软件侧只分配“元素数 ＝C×H×W”的扁平 int16 缓冲，不重排布局；真正的四通道打包（[C/4, H, W, 4]，lane0 落在 AXI 最低 16 位）由 IP 的写回决定。

核调用是“写寄存器—置 start—轮询 done”的同步模式，超时 5 s；VDMA 单帧等待超时 2 s；OV5640 自动对焦等待超时 2 s；I2C 在 EAGAIN/EBUSY 时重试 5 次、间隔 10 ms。所有超时都导向同一个结果：置故障位、保留缓冲与锁、拒绝后续调用，而不是重试或复位。这样做的原因很直接——DMA 与 IP 的状态一旦错位，重试只会把错误扩散到下一次调用；保留现场反而能让人从日志里定位问题。

启动与停止的顺序也有约束：采集前必须先写 VDMA 的行长、步长、地址与垂直尺寸，再置运行位触发；采完先停通道再 invalidate；显示要在写完整帧后 flush，再让 MM2S 读取。任何一步提前或滞后都会表现为“画面花屏但程序不报错”，这类问题的排查成本远高于一次显式的超时。

第三部分　完成情况及性能参数

3.1 整体介绍

作品已完成的工作可以概括为“一套算法、两套软件、三层产物”。算法是折叠倒残差的定点网络；软件是 PYNQ（Python/Linux）与 Vitis（裸机 C）两套实现，二者共用同一套寄存器契约与调用顺序；产物是四个 HLS IP、整机 bit/XSA/HWH，以及可复现的构建与校验脚本。

界面是作品最直观的成果。图 6 是 800×480 屏幕的渲染结果：顶栏是标题与“PYNQ + FPGA”标识，左上是 336×336 的相机窗口（由 224×224 最近邻放大），右上结果卡显示最近一次识别与耗时，下方两枚统计卡显示推理时间与已保存样本数，底部三个按钮分别为识别、补光与保存。图中状态为识别完成（Leaf blast、128 ms、已保存 12 帧），是界面渲染预览中的示例结果。

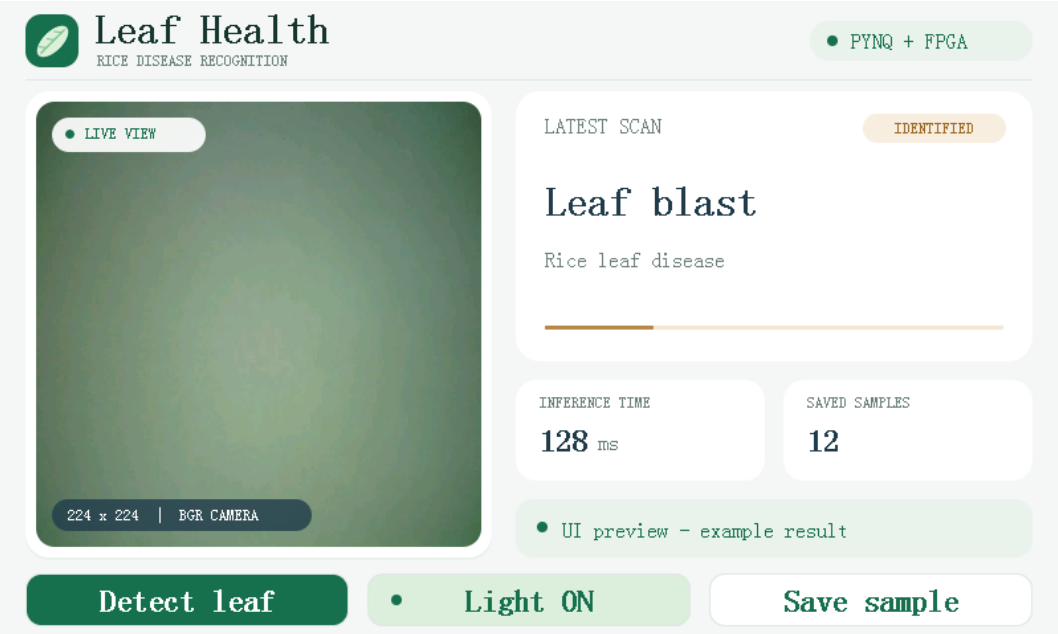


图 6　800×480 触摸屏界面（识别完成状态，渲染预览）

图 7 是板上相机预览的实拍截图，可以看到实时画面进入相机窗口，状态条显示 Ready。

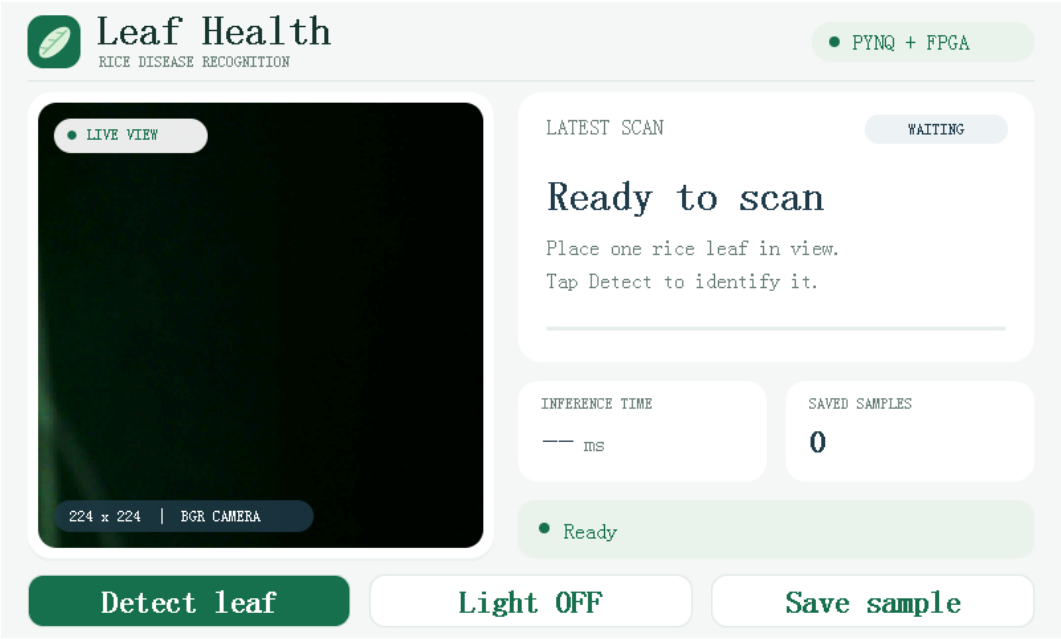


图 7　板上相机实时预览（OV5640，224×224 采集后放大至 336×336）

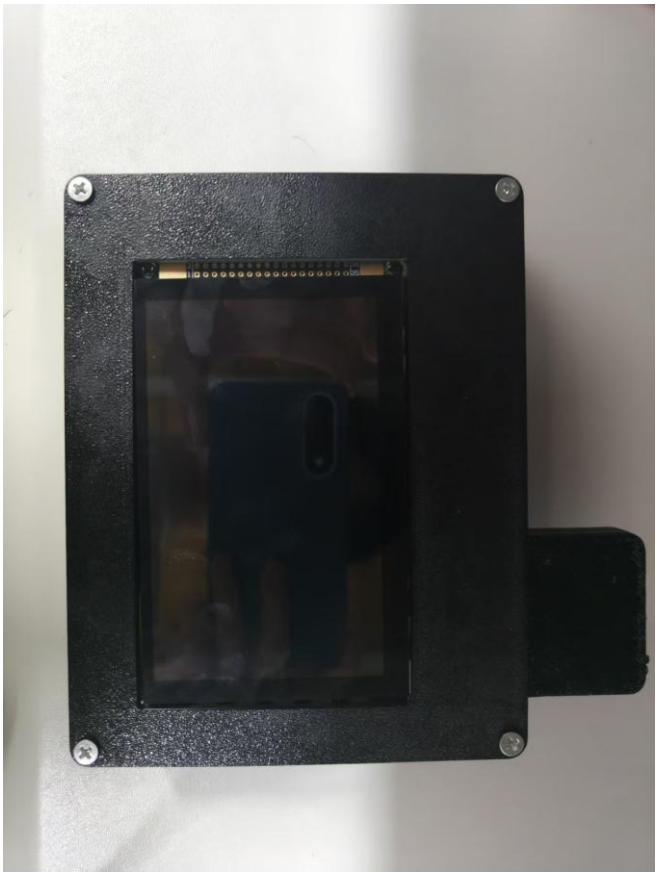


图 8　整机实物（正面：LCD 与相机窗口）



图 9　整机实物（侧 45°，可见外壳、屏幕与一侧接口）



图 10　整机实物（另一侧 45°，可见板载接口与侧面开窗）

3.2 工程成果

机械部分就是 2.2 节的外壳：3D 打印上下壳，正面开 LCD 窗、侧面留网口与 HDMI 开口，实物见图 8、图 9。

整机与裸机产物均由 2025.2 工具链生成（表 7）。2026-10-24 的板级实测确认四条链路同时可用：OV5640 出图（0x11001）、LCD 显示（0x11000）、GT911 触摸（0x8140 连读 100 次返回 911）、106 个权重数组加载完成；触摸曾因设备树把 MIO0 强制三态而 EAGAIN，改 pinctrl overlay 后恢复，未改 BOOT.BIN 与电压。

表 7　本轮重建的硬件产物

产物	说明
leaf.bit	含四核整机位流，由 write_bitstream 生成
leaf.xsa	固定平台且包含位流
design_1.hwh	与 bit 配对，PYNQ 2.7 可离线解析
routed dcp	布线后检查点，用于复算时序
trash_app.elf	裸机应用，入口 0x100000，末址 0x11340550
fsbl.elf	FSBL 启动加载器
boot.bin	FSBL→PL→APP 三段式启动镜像

板级记录方面，2026-10-24 的实测确认了四条链路同时可用：OV5640 出图（VDMA 状态 0x11001）、LCD 显示（MM2S 状态 0x11000）、GT911 触摸（读 0x8140 连续 100 次返回 911，约 0.061 s）、以及 106 个权重数组全部加载。触摸曾因设备树把 MIO0 强制为三态（0x1601）而持续 EAGAIN，改为 pinctrl overlay（I2C0 上拉、MIO0 禁止偏置）并解绑重绑后恢复，全过程未改 BOOT.BIN 与电压。

软件交付两套实现与一组校验入口：PYNQ 侧是一键/分步 Notebook、lanuch CLI 与 infer_file；裸机侧是 Vitis 2025.2 的 platform/BSP/FSBL/APP 与打包脚本（表 8）。

表 8　主要交付入口

入口	用途
00_one_click_start.ipynb	一键启动演示

入口	用途
01_bringup_and_demo.ipynb	分步调试（相机/LCD/触摸/推理）
lanuch.py --check	只读自检，不下载 PL
infer_file.py	固定 Q10 输入推理
sim/run_host_tests.py	主机回归
build/check_project.py	提交包完整性校验
validate_hls_rtl.py --extended	四核 RTL 功能验证

3.3 特性成果

表 9 把作品承诺的功能逐条对应到可判定的证据。

表 9 功能项与验证证据

功能	实测/验证结果
六分类推理	固定输入 10 次输出一致（−16010/7892/−4407/−3633/2183/−8760），判为第 1 类
单帧推理耗时	中位 278.7 ms，P95 279.9 ms，3.59 次/s
相机采集	224×224 BGR888，单帧 133.0 ms，状态 0x11001
LCD 显示	800×480，全帧拷贝与刷新 2.47 ms，状态 0x11000
触摸交互	0x8140 连读 100 次返回 911，配置头 800×480/5 点
样本保存	同帧写出 BGR、Q10 与 JSON 元数据
四核 RTL 功能	11 个用例、521,382 个 Q10 值逐值一致
整机实现	bit/xsa/hwh 生成，已约束路径 0 违例

六类的索引与标签在训练、权重文件名、硬件输出与界面文案之间必须一致，任何一处错位都会表现为“能跑但总判成某一类”。本作品固定为：0 healthy、1 bacterial_leaf_blight、2 brown_spot、3 leaf_blast、4 leaf_scald、5 narrow_brown。判定规则是取 6 个原始 Q10 分数的最大值，若小于 4000（约 3.906 个真实单位）则判为“无叶片”。

3.4 优化过程与优化前后对比

优化分三轮。界面：把整屏重绘改为状态缓存 + 24 KB 逐像素查表 + 只刷相机矩形，单帧 1103.6 ms→38.2 ms，采集—绘制—显示循环 0.584→7.50 FPS。采集：三组 A/B 中帧计数复位仅 7.527 vs 7.507 FPS（在噪声内），跳过

每帧复位第 8 帧报 0x14810，两项均未部署。工具链：四核由 2019.2 迁到 2025.2 重综合，整机 LUT 43,014→38,428、BRAM 137→111.5、DSP 143→134，已约束路径 WNS +0.207→+0.530 ns。

表 10 优化前后的性能与资源对比

指标	优化前	优化后	变化
界面绘制（单帧）	1103.6 ms	38.2 ms	约 29×
采集—绘制—显示循环	1711.3 ms / 0.584 FPS	133.2 ms / 7.50 FPS	约 12.8×
整机 LUT / BRAM	43,014（80.9%）/ 137	38,428（72.2%）/ 111.5	−10.7% / −18.6%
已约束路径 WNS	+0.207 ns	+0.530 ns	改善 0.323 ns

第四部分 总结

4.1 可扩展之处

一是精度与模型：当前网络在固定输入上输出稳定，但尚未在独立测试集上评估；后续可在保留 63 次调用结构的前提下替换模型（例如加入轻量注意力，或重新训练并采用量化感知训练），并补齐数据集的采集与标注。二是实时性：逐点卷积占 PL 时间的九成，把相邻层同尺寸特征留在片上、调整通道分块与突发长度都是可行方向；主机侧可把每层的多次寄存器写合并提交、减少轮询次数。两者都要改 IP 与驱动，收益需实测确认。三是工程化：补充摄像头像素时钟约束与板上波形、加入多帧环形缓冲以支持连续采集、把 4 个核封装为可参数化的 IP 生成流程。四是产品化：加入本地样本回流与增量校准、把裸机分支做成开机自启的独立固件。

4.2 心得体会

这个作品真正花时间的部分，不是把网络跑起来，而是搞清楚“什么算做出来了”。

第一件事是学会给结论标级别。我们一度把 2019.2 的历史构建记录当成了本轮成果来说明，直到逐条核对文件时间与验证记录里的 board_tested=false 才发现不对：那批数字确实是真的，但不是这一轮产生的。后来我们给每份结论都打上“本轮实测 / 历史记录 / 未验证”三种标签，并在报告里单列未验证清单。这个习惯让可复现性变成了可以检查的东西。

第二件事是理解“失败日志比成功日志更值钱”。四个核在 2025.2 上重建时，真正卡住我们的是四处具体的差异：一处 DATAFLOW 被前端判定为跨迭代读（HLS 200-994）、一处静态 burst 被 padding 条件挡住、一处动态二维 memcpy 只写了第 0 行偏置、一处偏置容量 1024 覆盖不了 1280 通道。这四条没有一条能靠读文档想到，全是被编译器的报错和环境里的哈希校验逼出来的。所以第六部分把它们写成了技能包里的“已知失败条件”，而不是藏在提交记录里。

第三件事是对“并行”的理解变了。我们一开始以为并行就是多开几个任务，后来发现真正的并行是“把可以独立验证的部分切出来”——原始应用与硬件接口、目标板资料、赛题与技能包这几条线可以同时取证，因为它们各自的通过判据互不依赖；而“先跑起来再说”的并行只会把错误叠在一起。同样的道理也适用于硬件：四个 IP 按卷积形态划分，而不是按层划分，才让每一层都能被单独验证。

第四件事是承认边界。这个作品目前能在板上跑通采集、显示、触摸与推理，固定输入 10 次输出逐位一致，但我们没有测过数据集准确率，没有测过功耗，也没有在同一输入上比过 ARM 与 PL 的耗时。这些空白不是没时间填，而是我们不愿意用“看起来合理”的数字去填。写报告时反复出现的“未验证”三个字，一开始让人觉得心虚，后来反而成了最踏实的部分——因为它明确告诉下一位接手的人，从哪里继续。

最后是对工具链的敬畏。一次 UTF-8 解码错误让 Vitis 的 platform 构建“退出码 0 但没有完成标记”，一次残留的 workspace 锁让 app 构建直接失败，一次 Python 解释器被工具自带的 stdlib 抢占让批处理脚本行为异常。这些都不是算法问题，却足以让一个“已经跑通”的工程在新机器上完全无法复现。第七部分之所以写得像操作手册，正是因为我们在这些坑里待过：可复现不是文档的美德，而是工程的底线。

第五部分　大模型协作记录

本部分记录大模型（含智能体）在本作品中的实际介入方式：任务如何下

发、模型给出了什么判断、哪些判断被证据推翻并纠正、以及这些纠正如何被固化成规则。

5.1 协作模式与智能体工作流

协作不是“问一句、抄一段”，而是先把工程拆成可独立验证的几条线，再分头取证、回到同一套门禁上验收。图 11 是这个闭环。按工程内的协作记录：主代理协调文件恢复、目标板资料与移植实现；并行代理分别审计原始应用与硬件接口、目标板 PYNQ 资料、竞赛要求与技能包，交互记录随主聊天一并导出。



图 11　协作与验证闭环：任务分解、并行取证、门禁验证、技能沉淀

5.2 原始提示词与任务下发

最初的任务由用户以自然语言下发，原文如下：

> 这是我们的工程 是用 vitis 做的 需要你根据资料以及其他你能在全网学到的任何东西 把我们的工程完美的移植到 pynq
>
> ——用户原始提示词，记录于 pynq/report/from_tar/collaboration_record.md

> 我们是自主选题赛道初级组 你也需要明确我们的赛题要求 然后开始工作
>
> ——用户第二条提示词，记录于同一文件

这两句话确定了三件事：目标是把 Vitis 裸机工程迁移到 PYNQ；资料范围包括本地附件与公开资料；但必须先弄清赛题要求再动手。后续工作以“工作单”形式下发，例如有一份工作单的标题是“按 README 重新还原、编译、导出”，正文明确了输入目录、输出目录、不得复用已有编译结果、以及“只调整本机路径”的边界。表 11 汇总了各阶段的下发方式与产出。

表 11　各阶段任务下发与产出

阶段	下发方式（要点）	模型侧产出
规则核对	要求先明确赛题要求、组别与提交物	赛题条款核对表、页码定位、缺证据清单
迁移实现	把 Vitis 工程迁移到 PYNQ，保留原 HLS	模块划分、寄存器映射、主机回归与量化约定
板级联调	解决相机、触摸、界面问题	A/B 实验、逐像素回归、pinctrl 修复方案
工具链升级	升级到 2025.2，不得改算法	四核编译副本差异说明、整机重建、时序与资源报告
重新还原	按 README 从对象快照还原、编译、导出	还原脚本、批处理日志、失败恢复脚本
本报告	依据评分要点与模板撰写技术报告	章节结构、图表、数据溯源与未验证清单

5.3 模型回答与关键决策

模型的价值不在于给出方案，而在于把“观察到的事实”和“由此得到的决策”分开写清楚。表 12 是其中五条，每条都附了当时的证据与验证范围。

表 12　关键决策记录（观察—证据—决策—范围）

观察	证据	形成的决策
裸机软件栈较旧	webtalk 统计含 Vitis v2019.2，驱动头部同版本	记录源版本；不得把提取 XSA 说成新版重建
存在 1.8 V 与 3.3 V 两套硬件	两个 XSA 与两套 bit/HWH，哈希不同	按同一 XSA 成套提取；禁止混配
目录名含 Trash	mess.c 的活动标签是六类水稻病害	按水稻病害保持接口；保留历史名为证据
原应用从外部读权重	camera_detection.c 调用 ReadParamq("trash_weight\\")	权重未齐前不伪造推理结果
裸机与 Linux 内存模型不同	地址契约与 PYNQ allocate 文档对照	用连续设备缓冲 + 显式缓存同步

5.4 自我纠错轨迹

协作记录里有一句必须保留的自我限定：“以上是实际观察引起的判断收敛与错误预防，不是伪造的‘模型先出错、后纠正’对话。”本节的七条纠错都对应记录中的真实观察或真实差异，逐条给出触发信号、根因、处理与复验方式。

表 13　自我纠错轨迹

#	触发信号	根因	处理与复验
1	离线测试通过被当成板上可用	验证级别混淆	引入“本轮实测/历史记录/未验证”三级标注，板级结论只认上板记录
2	官网访问超时，检索到第三方转录	想用二手资料补缺口	保留未核验状态，不用第三方转录替代官方细则
3	Notebook 重复初始化会清空句柄	防重入检查误放在生成器脚本顶层，而首单元会先清空句柄	把检查移入首代码单元第一条语句；新增 11 个状态不被重置的执行测试
4	固定输入 CLI 在加载 overlay 前分配权重失败	PYNQ 2.7 要求先建立 overlay 的内存元数据再分配	改为无 CMA 预校验→下载位流→再校验并分配；新增 3 个回归用例
5	Q10 输入读取长度偶发不一致	NumPy fromfile(int16) 会忽略奇数尾字节	要求输入与黄金值都校验精确字节长度（301,056 B）
6	Vivado 首轮批处理退出时崩溃	两个 intercon 子任务 EXCEPTION_ACCESS_VIOLATION，其余 37 个子任务完成	写恢复脚本只重置失败项与被阻塞的顶层综合，复验后完成
7	Vitis 构建“退出码 0 但没有完成标记”	控制台编码（0xd5）与残留 workspace 锁、残留 RigelApp/java 进程	局部隐藏子控制台 + chcp 65001；按日志精确清理残留进程；第三次通过

第六部分　技能包的提炼过程

技能包的目标很朴素：让一支从没接触过这个工程的队伍，照着文档就能把同样的东西做出来，并且在出问题时知道该停在哪一步。三个技能包分别对应三条主线的完整流程，都放在各工程的 skill/ 目录下，由 skill.md 与 README.md 两个文件构成。

6.1 三个技能包

表 14　三个技能包的定位与内容

技能包	适用场景	核心内容	声明的失效条件
hls-fixed-point-reproduction	从对象快照恢复 HLS 源码并重建四个定点核	六步执行顺序、17 个输入文件的 SHA 校验、四核成功判据、RTL 功能验证入口、C 仿真/RTL 验证/综合/cosim 不可互替	不得以本机既有工程为输入；输出不得落提交树；核级验证不证明精度或上板；1280 通道存在差异
zynq-pl-ps-release	恢复 Vivado 工程并产出可交付的 bit/XSA/HWH	两条输入路线（重综合 vs 接入新 IP）不可混用；XILINX_LOCAL_USER_DATA=no；write_bitstream 100% 且 bit 非空；write_hw_platform -fixed -include_bit	只有 route_design Complete 不算完成；不得把已有 board/ 产物当本轮生成；不得覆盖 PYNQ 配对与 SD 启动

技能包	适用场景	核心内容	声明的失效条件
pynq-runtime-handoff	把 PYNQ 运行环境交给下一位使用者	SHA/bit-HWH 配对/电压/权重格式/六类顺序；改小写须同步三处键；4000 阈值的语义；先固定输入比六分数，再看相机与界面；DMA 超时保持缓冲与锁	位流不配、类别或布局变化、电气未知、无黄金输入、DMA 未确认停止时均失效

6.2 从失败与观察到规则

技能包里的每一条规则都能追到一次具体的失败或一次具体的观察。表 15 抽取了其中八条，左列是原始信号，右列是提炼后的规则与验证方式。

表 15　从失败/观察到规则的提炼

来源（失败或观察）	提炼出的规则	如何验证有效	边界
HLS 200-994：DATAFLOW 在 2025.2 被判跨迭代读	先按报错定位到唯一 pragma，再做受控替换并记录 diff	四核重建后 RTL 功能逐值通过	只适用于该版本的该处结构，换版本需重跑
静态 burst 被 padding 条件挡住	卷积类核应显式给 cache 指令，不能依赖自动推断	conv 401,408 值与参考一致	cache 深度增大可能挤占 BRAM
动态二维 memcpy 只写第 0 行偏置	凡偏置/参数复制，必须验证全部行而不仅是首行	八组 pwdwconv 用例覆盖不同通道组合	仅覆盖已测的通道组合
偏置容量 1024 覆盖不了 1280 通道	容量参数必须与末层实际通道对齐，并写进交付说明	末层 1280 通道用例通过	扩容后与旧位流不再逐输入等价
VDMA 跳过复位后第 8 帧报 0x14810	没有被数据支撑的“优化”不进主线，只进实验记录	三组 A/B 复测，基线可复现	不适用于连续帧 + 环形缓冲的设计
Xsim 在 $fatal 后仍可能返回 0	验证器必须同时检查通过标记与失败关键字，不能只看退出码	负例注入后退出码为 1	只对文本日志有效
Notebook 重复初始化清空句柄	防重入检查必须放在真正的入口第一条语句	11 个状态的执行测试	依赖单元 tag 的定位方式不变
权重改小写后查不到	改名必须同步 three-place：weight_specs、_weight 查询、manifest	主机回归 53 项通过	跨平台大小写敏感性不同

第七部分　参考文献

[1] Sandler M, Howard A, Zhu M, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 4510-4520.

[2] Howard A G, Zhu M, Chen B, et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications[J]. arXiv preprint arXiv:1704.04861, 2017.

[3] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//CVPR. 2018: 2704-2713.

[4] Qiu J, Wang J, Yao S, et al. Going Deeper with Embedded FPGA Platform for Convolutional Neural Network[C]//Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2016: 26-35.

[5] Zhang C, Li P, Sun G, et al. Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks[C]//Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2015: 161-170.

[6] Mohanty S P, Hughes D P, Salathé M. Using Deep Learning for Image-Based Plant Disease Detection[J]. Frontiers in Plant Science, 2016, 7: 1419.

[7] Sladojevic S, Arsenovic M, Anderla A, et al. Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification[J]. Computational Intelligence and Neuroscience, 2016: 3289801.

[8] AMD. Vitis High-Level Synthesis User Guide: UG1399[R]. 2025.2 版. 2025.

[9] AMD. Vivado Design Suite User Guide: Design Analysis and Closure Techniques: UG906[R]. 2025.2 版. 2025.

[10] AMD. Zynq-7000 SoC Technical Reference Manual: UG585[R]. 2018.

[11] Xilinx. AXI VDMA v6.3 LogiCORE IP Product Guide: PG020[R]. 2017.

[12] PYNQ Project. PYNQ Documentation: Overlay Design Methodology, Memory Allocation and DMA[EB/OL]. https://pynq.readthedocs.io/.

[13] Linux Kernel Documentation. I2C Device Interface[EB/OL]. https://www.kernel.org/doc/html/latest/i2c/dev-interface.html.